

Последовательности команд и транзакции Redis

Последовательности команд

В типичном сценарии использования хранилища Redis разворачивается в качестве удаленного сервера или кластера серверов. При этом все взаимодействие между хранилищем и бизнес-логиками происходит по сети и подвержено сетевым задержкам.

В процессе исполнения команды Redis сетевые задержки проявляются дважды: на фазе отсылки команды на сервер и на фазе приема ответа сервера, т.е. последовательное исполнение N команд включает $2*N$ сетевых задержек. Для борьбы с ними платформенные клиенты Redis предоставляют функциональность по исполнению последовательностей команд.

В этом сценарии все команды посылаются единой последовательностью на сервер и ответ сервера на них так же принимается единым потоком, что позволяет сократить количество задержек до двух вне зависимости от количества команд в последовательности.

Формирование последовательности команд

Последовательность формируется путем создания объекта последовательности и добавления в него команд с соответствующими аргументами. Порядок добавления команд определяет порядок их исполнения на сервере.

Python: [RedisCommandBatch](#)

```
1 batch = RedisCommandBatch()
2 batch.Set('ключ #1', 'значение') \
3     .Incr('ключ #2') \
4     .Del('ключ #3')
```

C++ асинхронный: [redis::AsyncCommandBatch](#)

```
1 redis::AsyncCommandBatch batch;
2 auto set_op = batch.AddCommand( redis::commands::Set{ L"ключ #1"_sv,
3     L"значение"_sv } );
4 auto inc_op = batch.AddCommand( redis::commands::Incr{ L"ключ #2"_sv }
5     );
6 auto del_op = batch.AddCommand( redis::commands::Del{ L"ключ #3"_sv }
7     );
```

Асинхронный клиент в ответ на команду, помещаемую в последовательность, возвращает соответствующий ей объект асинхронной операции [AsyncOperation](#), позволяющий ожидать и извлекать результат её исполнения.

C++ устаревший: [redis::CommandBatch](#)

```
1 Int64 inc_res;
2
3 redis::CommandBatch batch;
4 batch << redis::command::Set{ L"ключ #1", L"значение" }
5     << redis::command::Incr{ inc_res, L"ключ #2" }
6     << redis::command::Del{ L"ключ #3" };
```

Исполнение последовательности команд

Сформированная последовательность команд передается на исполнение клиенту.

Python: [RedisClient.SendCommandBatch](#)

```
1 client = RedisClient('Имя.Кластера', True)
```

```
2 client.SendCommandBatch(batch)
```

C++ асинхронный: [redis::AsyncClient::SendCommands](#)

```
1 redis::AsyncClient client{ redis::AsyncClient::FromClusterName,  
    L"Имя.Клстера"_s };  
2 auto batch_op = client.SendCommands( std::move( batch ) );
```

Асинхронный клиент возвращает объект асинхронной операции [AsyncOperation](#) для всей последовательности, так же позволяющий ожидать её исполнения и проверять его успешность.

C++ устаревший: [redis::Client::SendCommandBatch](#)

```
1 redis::Client client{ L"Имя.Клстера", true };  
2 client.SendCommandBatch( batch );
```

Извлечение результатов исполнения команд

Python: [RedisCommandBatch.GetResult](#)

```
1 set_res = batch.GetResult(0)  
2 inc_res = batch.GetResult(1)  
3 del_res = batch.GetResult(2)
```

При работе с клиентом на Python результаты команд могут быть извлечены из объекта последовательности.

C++ асинхронный: [redis::AsyncOperation::GetResult](#)

```
1 auto set_res = set_op.GetResult();  
2 auto inc_res = inc_op.GetResult();  
3 auto del_res = del_op.GetResult();
```

Результаты команд асинхронного клиента на C++ извлекаются из объектов асинхронных операций, выдаваемых при добавлении команды в последовательность.

C++ устаревший: [redis::Client::SendCommandBatch](#)

API последовательностей команд синхронного клиента записывает результат исполнения непосредственно в пользовательскую переменную, ссылка на которую принимается в качестве одного из аргументов команды.

Транзакции Redis

Redis поддерживает атомарное исполнение последовательностей команд, называемых Redis-транзакцией. Транзакция гарантирует, что никакой запрос от других клиентов не может вклиниться между командами транзакции, и что либо будут исполнены все команды транзакции, либо ни одной. А также транзакция предоставляет механику условного исполнения, позволяющую реализовать Check-And-Set логику.

Механика условного исполнения позволяет автоматически отменять исполнение атомарной последовательности команд по изменению значений отслеживаемых ключей. Рассмотрим на тривиальном примере:

```
1 WATCH mykey  
2 val = GET mykey  
3 val = val + 1  
4 MULTI  
5 SET mykey $val  
6 EXEC
```

Команда WATCH запускает механизм условного исполнения, помещая ключ *mykey* в список слежения. Далее выбирается текущее значение по ключу и увеличивается, а в рамках транзакции, ограниченной командами MULTI и EXEC оно выставляется искомому ключу. Если между исполнением команды WATCH и началом исполнения блока команд транзакции значение по ключу *mykey* изменяется (к примеру, другим клиентом), транзакция отменяется и все команды транзакции отбрасываются. Если же значение по ключу *mykey* остается неизменным все команды транзакции исполняются.

Следует отметить важную особенность транзакций Redis, отличающих их от традиционных транзакций БД: завершение исполнения команды из блока транзакции с ошибкой не влечет отмены транзакции, эффект предыдущих команд остается и выполнение последующих команд продолжается.

Подробное описание транзакций можно найти в официальной документации: [Transactions – Redis](#).

Отслеживание ключей

Набор ключей, чье изменение должно отменять транзакцию, передается с помощью команды Watch.

Python: [RedisClient.Watch](#)

```
client.Watch([ 'отслеживаемый ключ' ])
```

C++ асинхронный: [redis::AsyncClient::SendCommand](#)

```
client.SendCommand( redis::commands::Watch{ L"отслеживаемый ключ"_sv }
).GetResult();
```

C++ устаревший: [redis::Client::Watch](#)

```
client.Watch( { L"отслеживаемый ключ"_s } );
```

Формирование транзакции

Транзакции используют тот же механизм, что и обычные последовательности команд.

Исполнение транзакции

Сформированная транзакция передается на исполнение клиенту. В качестве статуса исполнения транзакции клиент выдает значение типа bool: true, если транзакция исполнена, false, если отменена по изменению отслеживаемых ключей.

Python: [RedisClient.SendAtomicCommandBatch](#)

```
executed = client.SendAtomicCommandBatch(batch)
```

C++ асинхронный: [redis::AsyncClient::SendCommandsAtomic](#)

```
1 auto transaction_op = client.SendCommandsAtomic( std::move( batch ) );
2 bool executed = transaction_op.GetResult();
```

Асинхронный клиент возвращает объект асинхронной операции [AsyncOperation](#) для всей последовательности, так же позволяющий ожидать её исполнения и проверять его успешность.

C++ устаревший: [redis::Client::SendAtomicCommandBatch](#)

```
bool executed = client.SendAtomicCommandBatch( batch );
```

Извлечение результатов исполнения команд

Аналогично обычным последовательностям команд.